

13_Milvus使用指南 大纲

1、安装Docker DeskTop	1
1.1 下载安装包	1
1.2 设置Proxy	1
1.3 设置国内镜像（备选）	1
2、安装Milvus	2
2.1 standalone.bat 方式（不推荐）	2
2.2 docker compose方式	3
3、测试前准备工作	4
3.1 确保Milvus已启动	4
3.2 安装Milvus Python SDK	4
3.3 Milvus数据模型	5
3.3.1 整体结构	5
3.3.2 具体说明	5
4、基本用法	5
4.1 DDL操作	5
4.1.1 Database	5
1) 查看数据库	5
举例1：操作客户端	5
举例2：列出所有数据库	6
2) 创建新数据库	6
3) 删除数据库	6
4.1.2 Collection相关操作	6
1) 切换数据库	6
2) 查看数据库下的Collections	6
3) 创建collection	6
4) 删除collection	7
4.2 DML操作	7
4.2.1 嵌入模型初始化	7
4.2.2 准备collection	7
1) 创建collection	7
2) 查看collection元数据	7
4.2.3 准备数据	8
1) 准备原始数据	8
2) 生成嵌入向量	8
3) 查看生成的嵌入向量	9
4) 封装为可以插入的数据格式	9
4.2.4 写入数据	9
1) 写入数据	9
2) 手动flush	9

3、DQL操作	10
3.1 扫描数据	10
3.2 通过主键查询数据	10
3.3 相似度检索	11
1) 准备查询嵌入	11
2) 检索	11

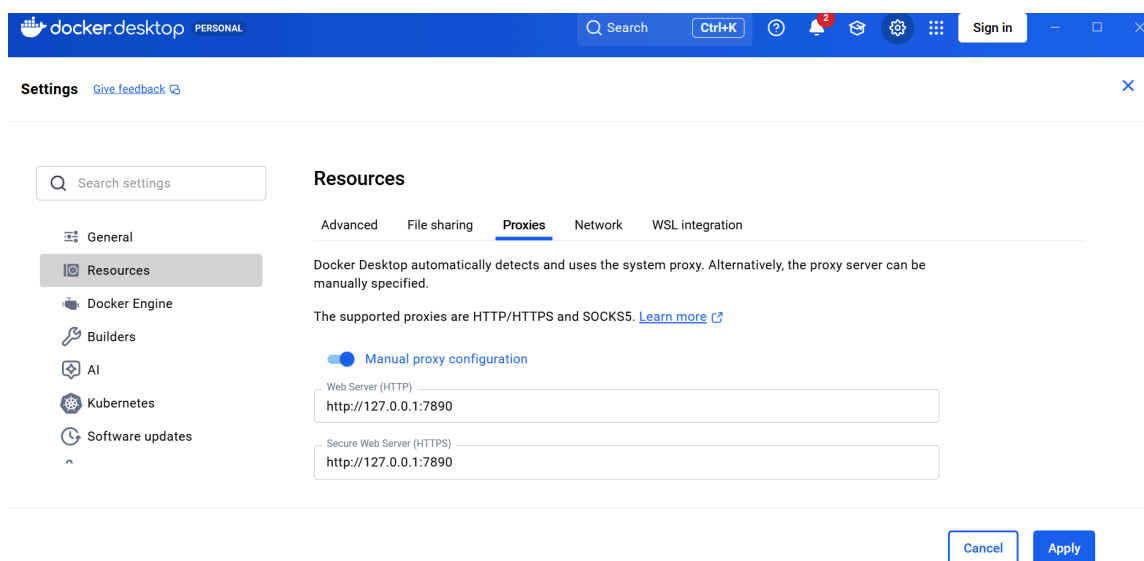
1、安装Docker DeskTop

1.1 下载安装包

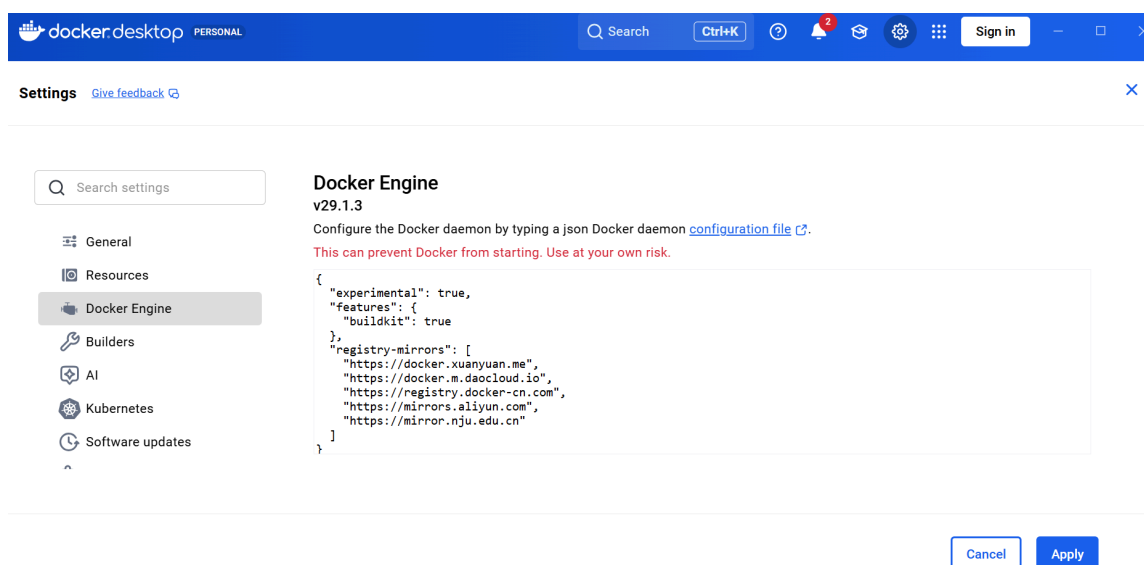
官网 <https://www.docker.com/>

按照指示下载安装

1.2 设置Proxy



1.3 设置国内镜像（备选）



南京大学镜像站：<https://docker.dockerimage.jobcher.com>

阿里云个人专属加速器（免费稳定）

- 登录 <https://cr.console.aliyun.com/cn-hangzhou/instances/mirrors> 左侧导航栏选择
镜像工具->镜像加速器
然后配置到上面的截图中

2、安装Milvus

2.1 standalone.bat 方式（不推荐）

新建Milvus文件夹

cd到文件夹

cmd 输入curl -o standalone.bat https://raw.githubusercontent.com/milvus-io/milvus/refs/heads/master/scripts/standalone_embed.bat

```
C:\Users\huan.zheng\Milvus>curl -o standalone.bat https://raw.githubusercontent.com/milvus-io/milvus/refs/heads/master/scripts/standalone_embed.bat
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100    4390  100    4390    0     0    1823      0  00:02    00:02    00:02     0

C:\Users\huan.zheng\Milvus>standalone.bat
Please use standalone_embed.bat restart|start|stop|delete

C:\Users\huan.zheng\Milvus>
```

首次输入 standalone.bat start

有可能docker镜像不对，要调整镜像，下载很慢，需要科学上网

```
C:\Users\huan.zheng\Milvus>standalone.bat start
Unable to find image 'milvusdb/milvus:v3.0.0' locally
v3.0.0: Pulling from milvusdb/milvus
4f4fb700ef54: Already exists
7646c8da3324: Downloading [=>] 1.049MB/29.53MB
467d5407c4fa: Downloading [>] 7.34MB/511.2MB
7bda03995917: Download complete
dfc2f426ddc1: Downloading [>] 1.049MB/80.08MB
f6456302a9ab: Downloading [===>] 2.097MB/26.69MB
396ad9033247: Download complete
c017c6f12b16: Download complete
2bd4a5ca11c1: Downloading [>] 2.097MB/431MB
f9caf8d9fbd3: Download complete
```

再次启动

输入 standalone.bat start

```
C:\Users\huan.zheng\Milvus>standalone.bat start
Wait for Milvus Starting...
Start successfully.
To change the default Milvus configuration, edit user.yaml and restart the service.
```

输入 .\standalone.bat stop 停止容器

docker desktop也能操作

2.2 docker compose方式

启动 3 个容器：`milvus-standalone` 主程序、`milvus-etcd` 元数据、`milvus-minio` 对象存储Milvus

1、PowerShell 新建一个空文件夹，下载 yml

输入运行：`Invoke-WebRequest https://github.com/milvus-io/milvus/releases/download/v3.0.0/milvus-standalone-docker-compose.yml -OutFile docker-compose.yml`

2、启动

`docker compose up -d`

3、查看状态

`docker compose ps`

✅正常输出：`milvus-standalone` 的 PORTS 列显示 `0.0.0.0:19530->19530/tcp, 0.0.0.0:9091->9091/tcp`

4、查看日志（等待服务完全就绪，需要 60-90 秒）

看到 `server started` 才算 Milvus 服务就绪，不是容器 Running 就可以连。

5、停止 / 销毁

powershell输入运行：

停止，保留数据

`docker compose stop`

销毁容器+保留volumes数据

`docker compose down`

销毁容器+删除全部数据

`docker compose down -v`

数据持久化：当前目录 `volumes/` 下面保存 etcd、minio、milvus 全部数据。
修改配置：复制 `milvus.yaml` 到本地，在 compose 里挂载；不要直接修改容器内部文件。

校验端口映射（必做） `docker port milvus-standalone`

有输出代表端口映射成功；空白 = 绑定失败，该容器不可用，直接删除重建。

docker desktop上可以看到容器

The screenshot shows the Docker Desktop interface. On the left is a sidebar with navigation options: Ask Gordon, Containers (selected), Images, Volumes, Kubernetes, Builds, Models, MCP Toolkit, Docker Hub, Docker Scout, and Extensions. The main area is titled 'Containers' and shows overall usage: 14.95% / 1200% (12 CPUs available) for CPU and 304.38MB / 15.08GB for memory. Below this is a table of running containers:

	Name	Container ID	Image	Port(s)	CPU (%)	Memor	Actions
☐	milvus	-	-	-	11.11%	302.88M	
☐	minio	bd5ea2b61382	minio/minio	9000:9000 Show all ports (2)	0.42%	104.2M	
☐	etcd	2c68921a4816	coreos/etcd	-	1.48%	15.48M	
☐	standalone	5b0d03022c0e	milvusdb/n	19530:19530 Show all ports (2)	9.21%	183.2M	

Python 连接测试

3、测试前准备工作

3.1 确保Milvus已启动

```
C:\Users\huan.zheng\Milvus>standalone.bat start
Wait for Milvus Starting...
Start successfully.
To change the default Milvus configuration, edit user.yaml and restart the service.
```

The screenshot shows the Docker Desktop interface with the 'Containers' tab selected. The main area displays the 'milvus-standalone' container, which is running. The table of containers is as follows:

	Name	Container ID	Image	Port(s)	CPU (%)	Memor	Actions
☐	milvus-standalone	d2c7060ff7d7	milvusdb/n	19530:19530 Show all ports (3)	12.16%	205.3M	
☐	langfuse	-	-	-	0%	0B / 0B	
☐	docker	-	-	-	0%	0B / 0B	

3.2 安装Milvus Python SDK

```
pip install pymilvus
```

Milvus Lite已包含在Milvus的Python SDK中，此处不用Milvus Lite 直接访问在docker desktop安装的、Standalone模式的Milvus服务。

3.3 Milvus数据模型

3.3.1 整体结构

Milvus数据模型层级：database>collection>partition>entity

- database Milvus的数据库，用来隔离不同业务数据
- collection 最核心的逻辑容器，类似关系型数据库里的table
- partition 分区，是collection的子集，不是必须手动创建，一个collection至少会有默认partition
- entity 可以理解为collection中的一条记录，类似关系型数据库的一行数据。

3.3.2 具体说明

向量数据库存储特征向量和对应的文本，为了将不同用途的向量分类别地存放，需要创建不同的collection。

比如COLLECTION_NAME = "docs",用于存放切分好地文档片段向量

4、基本用法

4.1 DDL操作

4.1.1 Database

1) 查看数据库

举例1：操作客户端

```
In [20]: from pymilvus import connections
try:
    connections.connect(
        alias="default",
        host="localhost",
        port="19530",
        # user="root", password="xxx" # 开启认证就打开
    )
    print("✅ Milvus连接成功")
    connections.disconnect("default")
except Exception as e:
    print(f"❌ 连接失败: {e}")
```

✅ Milvus连接成功

```
C:\Users\huan.zheng\AppData\Local\Temp\ipykernel_22168\4254525221.py:3: PyMilvusDeprecationWarning: `connections.connect` is an ORM-style PyMilvus API and will be removed in PyMilvus 3.1. Use `MilvusClient` instead.
connections.connect(
C:\Users\huan.zheng\AppData\Local\Temp\ipykernel_22168\4254525221.py:10: PyMilvusDeprecationWarning: `connections.disconnect` is an ORM-style PyMilvus API and will be removed in PyMilvus 3.1. Use `MilvusClient` instead.
connections.disconnect("default")
```

```
In [21]: from pymilvus import MilvusClient

client = MilvusClient("http://localhost:19530")
```

举例2：列出所有数据库

```
In [26]: existed_databases = client.list_databases()

for db in existed_databases:
    print(db)
```

default

2) 创建新数据库

```
In [23]: db_name = "rag_demo"
if db_name not in existed_databases:
    client.create_database(db_name=db_name)
```

3) 删除数据库

如果数据库下有Collection则无法删除，需要先删除它所有的Collection才能删除Database

```
In [25]: client.drop_database(db_name=db_name)
```

4.1.2 Collection相关操作

1) 切换数据库

```
In [7]: client.use_database(db_name=db_name)
```

2) 查看数据库下的Collections

```
In [29]: collections = client.list_collections()

for coll in collections:
    print(coll)
```

3) 创建collection

```
In [30]: collection_name = "docs"
```



```
client.create_collection(
    collection_name=collection_name,
    dimension=1024, # 和向量化时嵌入模型一致
    metric_type="COSINE"
)
```

4) 删除collection

```
In [10]: client.drop_collection(collection_name=collection_name)
```

4.2 DML操作

4.2.1 嵌入模型初始化

```
In [31]: import os
from langchain_openai import OpenAIEmbeddings
from dotenv import load_dotenv

load_dotenv()
# =====
# 3. 初始化 Embedding 模型
# =====
embed_model = OpenAIEmbeddings(
    model="Pro/BAAI/bge-m3",
    openai_api_base=os.environ["SILICON_BASE_URL"],
    openai_api_key=os.environ["SILICON_API_KEY"],
    # dimensions=1024, # 可保留；最关键的是 collection 要按 1024 建
)
```

bge-m3 是固定 1024 维的模型，SiliconFlow 的接口不接受 dimensions 这个参数

4.2.2 准备collection

1) 创建collection

```
In [ ]: client.drop_collection(collection_name="docs0826")
```

```
In [32]: collection_name = "docs0825"

client.create_collection(
    collection_name=collection_name,
    dimension=1024, # 要和嵌入模型的dimensions一致
    metric_type="COSINE"
)
```

2) 查看collection元数据

```
In [33]: from rich import print as rprint
metadata = client.describe_collection(collection_name=collection_name)
```

```
rprint(metdata)
```

```
{
  'collection_name': 'docs0825',
  'auto_id': False,
  'num_shards': 1,
  'description': '',
  'fields': [
    {
      'field_id': 100,
      'name': 'id',
      'description': '',
      'type': <DataType.INT64: 5>,
      'params': {},
      'is_primary': True
    },
    {
      'field_id': 101,
      'name': 'vector',
      'description': '',
      'type': <DataType.FLOAT_VECTOR: 101>,
      'params': {'dim': 1024}
    }
  ],
  'functions': [],
  'aliases': [],
  'collection_id': 468638891251865452,
  'consistency_level': 2,
  'consistency_level_name': 'Bounded',
  'properties': {'timezone': 'UTC', 'namespace.sharding.enabled': 'f'},
  'num_partitions': 1,
  'enable_dynamic_field': True,
  'enable_namespace': False,
  'schema_version': 0,
  'created_timestamp': 468639143640694803,
  'update_timestamp': 468639143640694803
}
```

4.2.3 准备数据

1) 准备原始数据

```
In [34]: texts = [
    "database Milvus的数据库，用来隔离不同业务数据",
    "collection 最核心的逻辑容器，类似关系型数据库里的table",
    "partition 分区，是collection的子集，不是必须手动创建，一个collection至少会有默认partition",
    "entity 可以理解为collection中的一条记录，类似关系型数据库的一行数据。"
]
```

2) 生成嵌入向量

```
In [35]: vectors = embed_model.embed_documents(texts)
```

嵌入模型不能写dimensions 这个参数，否则会报错

3) 查看生成的嵌入向量

```
In [36]: print(len(vectors))

print(len(vectors[0]))

print(vectors[0][:5])

4
1024
[-0.019505193457007408, 0.030341412872076035, -0.03421149030327797, -0.01780235953629017, -0.012152045033872128]
```

4) 封装为可以插入的数据格式

```
In [37]: data = [
    {
        "id": i,
        "vector": vectors[i],
        "text": texts[i],
        "source": "demo"
    } for i in range(len(texts))
]
```

4.2.4 写入数据

1) 写入数据

```
In [38]: insert_res = client.upsert(
    collection_name=collection_name,
    data=data,
)

print("insert result :",insert_res)

insert result : {'upsert_count': 4, 'ids': [0, 1, 2, 3]}
```

2) 手动flush

Milvus 不会第一时间将数据落盘，要看写入效果，我们手动flush，将数据刷写到磁盘

```
In [39]: client.flush(collection_name=collection_name)
```

3) 查看collection统计信息

```
In [40]: stats = client.get_collection_stats(collection_name=collection_name)
print(stats)

{'row_count': 4}
```

3、DQL操作

3.1 扫描数据

```
In [41]: iterator = client.query_iterator(
        collection_name=collection_name,
        filter="",
        output_fields=["*"])

i=0
while True:
    rows = iterator.next()

    if not rows:
        break
    for row in rows:
        print(f"第{i + 1}条数据")
        # print(row)

        print(f"id = {row['id']}, vector = {row['vector'][:5]}, text = {row['text']}, source = {row['source']}")
        i += 1

iterator.close()
```

第1条数据

id = 0, vector = [-0.019505193457007408, 0.030341412872076035, -0.03421149030327797, -0.01780235953629017, -0.012152045033872128], text = database Milvus的数据库，用来隔离不同业务数据, source = demo

第2条数据

id = 1, vector = [-0.0023934897035360336, 0.010630395263433456, 0.032089270651340485, 0.004786979407072067, 0.043049801141023636], text = collection 最核心的逻辑容器，类似关系型数据库里的table, source = demo

第3条数据

id = 2, vector = [-0.011202643625438213, 0.036571212112903595, 0.0006369245238602161, -0.017851954326033592, 0.00404740683734417], text = partition 分区，是collection的子集，不是必须手动创建，一个collection至少会有默认partition, source = demo

第4条数据

id = 3, vector = [0.012002291157841682, 0.05517936497926712, -0.018782805651426315, -0.009897992946207523, 0.018782805651426315], text = entity 可以理解为collection中的一条记录，类似关系型数据库的一行数据。 , source = demo

3.2 通过主键查询数据

```
In [42]: res = client.get(
        collection_name=collection_name,
        ids=[0,1,2]
    )
print(len(res))

for i in range(len(res)):
```

```
print(f"第{i+1}条数据：")
print(f"id = {res[i]['id']}, vector = {res[i]['vector'][:5]}, text = {res[i]['text']}, source = {res[i]['source']}]")
```

3

第1条数据：

id = 0, vector = [-0.019505193457007408, 0.030341412872076035, -0.03421149030327797, -0.01780235953629017, -0.012152045033872128], text = database Milvus的数据库，用来隔离不同业务数据, source = demo

第2条数据：

id = 1, vector = [-0.0023934897035360336, 0.010630395263433456, 0.032089270651340485, 0.004786979407072067, 0.043049801141023636], text = collection 最核心的逻辑容器，类似关系型数据库里的table, source = demo

第3条数据：

id = 2, vector = [-0.011202643625438213, 0.036571212112903595, 0.0006369245238602161, -0.017851954326033592, 0.00404740683734417], text = partition 分区，是collection的子集，不是必须手动创建，一个collection至少会有默认partition, source = demo

3.3 相似度检索

1) 准备查询嵌入

```
In [43]: query = "什么是collection?"
query_vector = embed_model.embed_query(query)
```

2) 检索

```
In [50]: results = client.search(
    collection_name=collection_name,
    data=[query_vector],
    limit=3,
    output_fields=["text","source","id"]
)

for res in results[0]:
    print(res)
```

```
{'id': 2, 'distance': 0.6808801889419556, 'entity': {'id': 2, 'text': 'partition 分区，是collection的子集，不是必须手动创建，一个collection至少会有默认partition', 'source': 'demo'}}
{'id': 1, 'distance': 0.6578744649887085, 'entity': {'id': 1, 'text': 'collection 最核心的逻辑容器，类似关系型数据库里的table', 'source': 'demo'}}
{'id': 3, 'distance': 0.2471303641796112, 'entity': {'id': 3, 'text': 'entity 可以理解为collection中的一条记录，类似关系型数据库的一行数据。', 'source': 'demo'}}
```